

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43012

COURSE OUTCOME COVERED

CO-4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool : Pseudocode Writing Task (Algorithm Design)
Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I _____CH.Bhavya Sri/192411249_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

CH.Bhavya Sri

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. XML Parsing Algorithm

Algorithm: Parse XML Document

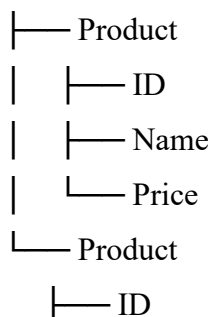
BEGIN

1. Start the program.
2. Load the XML document from the specified file.
3. Check whether the XML file is loaded successfully.
4. If loading fails, display "XML file not found" and STOP.
5. Create an XML parser.
6. Parse the XML document using the parser.
7. Access the root node of the XML document.
8. Traverse each child node of the root node.
9. For each child node:
 - a. Identify the node name.
 - b. Access its child elements.
 - c. Extract the required values.
 - d. If the node contains nested elements,
recursively traverse the nested nodes.
10. Store the extracted data.
11. Display the extracted information in a readable format.
12. Stop parsing after all nodes are processed.

END

Example XML structure:

Products



└─ Name
└─ Price

2. MVC Workflow

Algorithm: MVC Request–Response Workflow

BEGIN

1. Start the web application.
2. User sends a request from the browser.
3. Receive the request at the Controller.
4. Controller identifies the requested operation.
5. Controller sends the required request to the Model.
6. Model processes the business logic.
7. Model establishes connection with the database.
8. Execute the required database operation.
9. Retrieve the required data from the database.
10. Model sends the processed data back to the Controller.
11. Controller sends the data to the View.
12. View generates the required web page.
13. View creates the final response.
14. Send the response to the user's browser.
15. Display the result to the user.

END

MVC Flow:

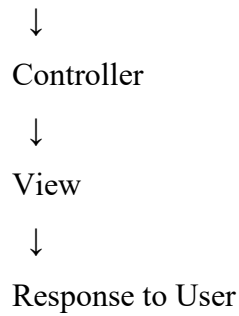
User



Controller



Model ↔ Database



3. JSP Request Flow

Algorithm: JSP Request–Response Flow

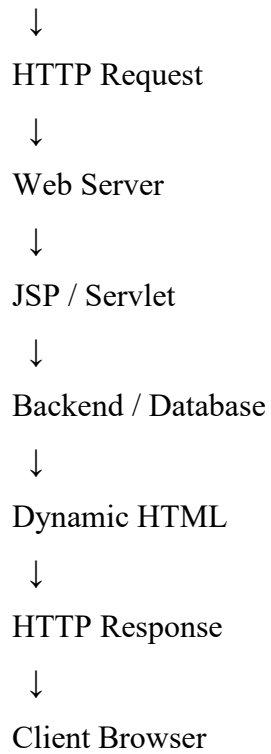
BEGIN

1. Start the JSP web application.
2. User enters a URL or submits a form.
3. Browser sends an HTTP request to the web server.
4. Web server receives the request.
5. Identify the requested JSP page.
6. If the JSP page is not compiled,
 translate JSP into a Servlet and compile it.
7. Execute the generated Servlet.
8. Servlet/JSP processes the user's request.
9. If required, communicate with backend components
 such as Servlets, Java classes, or databases.
10. Retrieve and process the required data.
11. Generate dynamic HTML content using JSP.
12. Web server sends the generated response to the browser.
13. Browser displays the dynamic web page to the user.

END

JSP Flow:

Client



4. Database Retrieval

Algorithm: Retrieve Data from Database

BEGIN

1. Start the application.
2. Load the required database driver.
3. Establish a connection with the database.
4. Check whether the connection is successful.
5. If connection fails, display an error message and STOP.
6. Create an SQL query.
7. Send the SQL query to the database.
8. Execute the query.
9. Receive the Result Set from the database.
10. Check whether records are available.
11. If records are available:
 - a. Read each record one by one.

- b. Extract the required fields.
 - c. Display the retrieved data.
12. If no records are found, display "No records found".
 13. Close the Result Set.
 14. Close the database connection.
 15. Stop the program.

END

Database Flow:

Application



Connect to Database



Execute SQL Query



Result Set



Process Records



Display Data



Close Connection

5. PHP Validation

Algorithm: Validate User Input Using PHP

BEGIN

1. Start the PHP program.
2. Receive the data submitted through the web form.
3. Read the Name, Email, and Password fields.

4. Remove unnecessary spaces from the input.
5. Check whether any required field is empty.
6. If a field is empty:
 Display "Field cannot be empty".
7. Validate the email format.
8. If the email format is invalid:
 Display "Invalid email address".
9. Check the password.
10. If the password does not satisfy the required length or format:
 Display "Invalid password".
11. Check whether any validation error exists.
12. If errors exist:
 Display the appropriate error messages.
 Do not submit the form.
13. Otherwise:
 Accept the input.
 Process/store the submitted data.
 Display "Form submitted successfully".
14. Stop the program.

END

PHP Validation Flow:

User Input



Check Empty Fields



Validate Email



Validate Password



Errors?



Yes

No



Show

Accept

Errors

Input



Submit Form